

Lab 12 · El trabajo que ocurre solo

Curso de Spring Boot · Servicio de Impuestos Internos · 2026

60 minutos · Spring Boot 4.1.0 · Java 25 (Temurin)

Resumen

Que hay trabajo que debe ocurrir sin que nadie lo pida, y trabajo que no debe hacer esperar a quien está en la ventanilla: **3,011 s el envío síncrono, 0,003 s el asíncrono**. Y el problema que nadie ve venir cuando la aplicación se despliega dos veces.

Índice

1. Antes de empezar	2
1.1. Qué vas a lograr	2
1.2. Qué necesitas tener listo	2
1.3. Cómo copiar el código de esta guía	2
1.4. La puesta a punto	3
2. El caso	3
2.1. El conserje de noche, que es la metáfora de este laboratorio	3
3. Los pasos	3
3.1. Paso 1 · Una tarea que corre sola	3
3.2. Paso 2 · Cron, y cómo se lee	5
3.3. Paso 3 · Que el usuario no espere	7
3.4. Paso 4 · Hilos virtuales	9
3.5. Paso 5 · El problema que nadie ve venir	10
4. Lo que aprendiste	11
5. Para profundizar	11
6. Antes de cerrar	11

1 Antes de empezar

1.1 Qué vas a lograr

Todo lo que tu aplicación hace, lo hace **porque alguien se lo pidió**. Hoy empieza a trabajar sola.

Vas a programar una tarea que corre cada pocos segundos, otra que corre a una hora exacta, vas a sacar del camino del usuario un trabajo lento —y a medir la diferencia—, y vas a cambiar el modelo de hilos del servidor con **una línea de configuración**. Y al final vas a ver el problema que tienen todas las tareas programadas y que sólo aparece **cuando la aplicación se despliega dos veces**.

1.2 Qué necesitas tener listo

Requisito	Cómo lo compruebas	Qué tiene que salir
Los labs 01 y 02 hechos	Sabes crear endpoints y qué es un bean	—
Estar en la carpeta del lab	cd labs/lab-12-tareas/practica	El cd no da error
Ninguna base de datos	Este lab no la usa	—

1.3 Cómo copiar el código de esta guía

Al copiar de un PDF se pierden los espacios del principio de línea, y a veces una línea larga se parte en dos. Con Java no importa. El código completo está en labs/lab-12-tareas/solucion/.

1.4 La puesta a punto

```
cd labs/lab-12-tareas/practica
./mvnw spring-boot:run
```

Escucha en el **8103**. **Déjala corriendo un rato**: en este lab lo interesante pasa **sin que pidas nada**, así que hay que mirar la consola en silencio.

2 El caso

La oficina de la DGT cierra a las ocho. Pero hay trabajo que se hace **cuando no hay público**: el cierre del día, los recordatorios, las notificaciones que nadie está esperando en el mostrador.

2.1 El conserje de noche, que es la metáfora de este laboratorio

Hay trabajo que se hace con la oficina cerrada, y trabajo que no debe hacerse en el mostrador.

El cierre nocturno. Todos los días, sin que nadie lo pida, alguien cuadra las cuentas. Y hay una decisión sobre cómo se programa que parece un detalle y no lo es:

- «*empieza cada 5 minutos*» — si un día el cierre tarda 6, **empieza el siguiente con el anterior a medias**. Dos personas cuadrando las mismas cuentas a la vez.
- «*empieza 5 minutos después de terminar el anterior*» — nunca se solapan. Es lo que se usa aquí.

El recordatorio de las nueve. Ése no es «cada tanto»: es **a una hora**. Y una hora sin decir de dónde —qué huso— es media hora de diferencia el día que el servidor esté en otro sitio.

Y el correo del mostrador. Cuando emites un trámite hay que avisar por correo. El correo tarda un segundo. **Multiplicado por tres destinatarios, el ciudadano espera tres segundos delante de la ventanilla** mirando cómo mandas correos. No hace falta: se apunta el encargo y se le atiende.

Eso último tiene un precio que hay que decir: **si el correo falla, el ciudadano ya se fue**.

3 Los pasos

3.1 Paso 1 · Una tarea que corre sola

Qué vamos a hacer

Encender la programación de tareas y escribir una que corre cada pocos segundos.

Para entenderlo mejor

El cierre nocturno. Nadie lo pide: ocurre.

El problema

Todo lo que has escrito hasta ahora se dispara con una petición HTTP. Un cierre diario no tiene petición: tiene **un reloj**.

La alternativa, y por qué no

- **Un cron del sistema operativo** que llame a un endpoint: funciona, y reparte la lógica entre el sistema y la aplicación — el día que alguien mire sólo el código, no verá que eso existe.
- **Un ScheduledExecutorService a mano**: control total, y hay que gestionar su ciclo de vida.
- **@Scheduled**, que es lo de aquí: la tarea vive junto al código que la hace, y el planificador lo pone Spring.
- **Quartz**, cuando haga falta persistir el plan o coordinar varias instancias. Es la respuesta al problema del paso 5, y es otra pieza que operar.

Y la decisión que da nombre al paso: `fixedDelay` y no `fixedRate`.

- `fixedRate = 5000` cuenta 5 s **desde que empieza** una hasta que empieza la siguiente. Si la tarea tarda más que el intervalo, **se solapan**.
- `fixedDelay = 5000` cuenta 5 s **desde que TERMINA** una. Nunca se solapan.

El día que el cierre tarde más de lo normal —porque hay más trámites que de costumbre, justo el día que más importa— `fixedRate` arranca otro encima. **`fixedDelay` es el valor por defecto correcto.**

Se pega

En `practica/src/main/java/cl/dgt/tareas/Lab12Application.java`, **arriba**:

```
import org.springframework.scheduling.annotation.EnableScheduling;
```

y la anotación **sobre la clase**, junto a `@SpringBootApplication`:

```
@EnableScheduling
```

Archivo **nuevo** `practica/src/main/java/cl/dgt/tareas/programadas/CierreNocturno.java` — entero:

```
package cl.dgt.tareas.programadas;

import cl.dgt.tareas.soporte.Instancia;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;

import java.time.LocalDateTime;
import java.util.concurrent.atomic.AtomicInteger;

@Component
public class CierreNocturno {

    private static final Logger log = LoggerFactory.getLogger(CierreNocturno.class);

    private final Instancia instancia;
    private final AtomicInteger vueltas = new AtomicInteger();

    public CierreNocturno(Instancia instancia) {
        this.instancia = instancia;
    }
}
```

```

@Scheduled(fixedDelay = 5000, initialDelay = 3000)
public void ejecutar() throws InterruptedException {
    int n = vueltas.incrementAndGet();
    log.info("[CIERRE] {} · vuelta {} · {} · hilo {}",
            instancia.nombre(), n, LocalTime.now().withNano(0),
            Thread.currentThread());
    Thread.sleep(1000);
}

public int vueltas() {
    return vueltas.get();
}
}

```

Lo que vas a ver

Sin llamar a nada, sólo esperando:

```

[CIERRE] instancia-8103 · vuelta 1 · 01:16:15
[CIERRE] instancia-8103 · vuelta 2 · 01:16:21
[CIERRE] instancia-8103 · vuelta 3 · 01:16:27
[CIERRE] instancia-8103 · vuelta 4 · 01:16:33
[CIERRE] instancia-8103 · vuelta 5 · 01:16:39

```

Mira los segundos: 15, 21, 27, 33, 39. Entre vuelta y vuelta pasan **6 segundos**, no 5.

Y ahí está el paso entero: la tarea duerme 1 segundo, y `fixedDelay` cuenta desde que **termina**. $5 + 1 = 6$. Con `fixedRate` habrían salido cada 5, y el día que la tarea tardara 6, dos a la vez.

Vas bien si...

Salen líneas `[CIERRE]` sin que hayas pedido nada, y entre una y otra pasan **6 segundos**.

Si te atascas

1 · No sale ninguna línea `[CIERRE]`.

Falta `@EnableScheduling` sobre la clase de arranque. **Sin ella, `@Scheduled` no hace nada y no avisa** — es el fallo más silencioso de este lab.

2 · Sale una sola vez y no se repite.

Escribiste `initialDelay` donde iba `fixedDelay`, o la tarea lanzó una excepción: si una ejecución falla, el planificador puede dejar de repetirla.

3 · Port 8103 was already in use

La tienes arrancada en otra terminal:

```
lsof -ti:8103 | xargs kill -9
```

3.2 Paso 2 · Cron, y cómo se lee

Qué vamos a hacer

Una tarea que corre **a una hora**, no cada tanto.

Para entenderlo mejor

El recordatorio de las nueve. No es «cada doce horas»: es **a las nueve**.

El problema

`fixedDelay` sirve para cadencias. No sirve para «todos los días a las 2 de la mañana», que es lo que pide casi cualquier proceso de cierre real.

La alternativa, y por qué no

`fixedDelay` cuando importa **el intervalo**; `cron` cuando importa **la hora**. No son intercambiables: un `fixedDelay` de 24 horas se va desplazando con cada reinicio.

Y la parte que hay que escribir siempre: la zona. Sin `zone`, la expresión se interpreta en la zona de la máquina — que en tu portátil es la de aquí y en el servidor suele ser UTC. Un «a las 2 de la mañana» que en producción ocurre a las 22:00 del día anterior es un error que **no se ve hasta que alguien mira los datos**. Y con horario de verano, la misma expresión salta una hora dos veces al año.

Once caracteres, y quita una clase entera de incidentes.

Se pega

Archivo **nuevo** `practica/src/main/java/cl/dgt/tareas/programadas/Recordatorio.java` — entero:

```
package cl.dgt.tareas.programadas;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;

import java.time.LocalDateTime;

@Component
public class Recordatorio {

    private static final Logger log = LoggerFactory.getLogger(Recordatorio.class);

    @Scheduled(cron = "*/10 * * * *", zone = "America/Santiago")
    public void avisar() {
        log.info("[CRON] recordatorio · {}", LocalDateTime.now().withNano(0));
    }
}
```

El cron de Spring tiene SEIS campos, no cinco. Segundo, minuto, hora, día del mes, mes, día de la semana. Casi todo lo que encuentres en internet es el cron de Unix, de cinco — y

al pegarlo aquí se desplaza todo un campo.

Lo que vas a ver

```
[CRON] recordatorio · 01:16:20
[CRON] recordatorio · 01:16:30
[CRON] recordatorio · 01:16:40
[CRON] recordatorio · 01:16:50
```

Segundos 20, 30, 40, 50: clavados. A diferencia del `fixedDelay`, aquí no importa cuánto tarde la tarea: la hora es la hora.

Vas bien si...

Las líneas `[CRON]` caen en segundos exactos múltiplos de diez, y las `[CIERRE]` no.

Si te atascas

1 · Cron expression must consist of 6 fields

Pegaste un cron de Unix, de cinco campos. Le falta el de los segundos delante.

2 · Corre, pero a una hora que no esperabas.

Falta `zone`, y se está usando la del servidor.

3.3 Paso 3 · Que el usuario no espere

Qué vamos a hacer

Sacar del camino del usuario un trabajo lento, y **medir** la diferencia.

Para entenderlo mejor

Apuntar el encargo y atender al siguiente. El correo se manda igual; lo que no se hace es **mandarlo con el ciudadano delante**.

El problema

Emitir un trámite manda tres correos, y cada uno tarda un segundo. El ciudadano espera tres segundos mirando cómo mandas correos que no le aportan nada.

La alternativa, y por qué no

- **Hilos a mano** o `CompletableFuture.runAsync`: funciona, y hay que gestionar el pool.
- `@Async`, que es lo de aquí: una anotación, y el método vuelve al instante.
- **Una cola de verdad** (RabbitMQ, Kafka): es lo correcto **en cuanto el trabajo no se pueda perder**. `@Async` vive en memoria: si el proceso se cae, lo que estuviera en vuelo desaparece. Un correo perdido se aguanta; un pago, no.

Y la trampa que hay que decir en voz alta: `@Async` funciona **por un proxy**. Llamar a un método `@Async` **desde otro método de la misma clase** no pasa por el proxy y **se ejecuta de forma síncrona, sin avisar**. Es el fallo número uno de `@Async` — y es la misma regla de `@Transactional`, por el mismo motivo. Por eso el trabajo vive en **otro bean**.

Se pega

Arriba, en la clase de arranque:

```
import org.springframework.scheduling.annotation.EnableAsync;
```

```
@EnableScheduling
@EnableAsync
@SpringBootApplication
```

En practica/src/main/java/cl/dgt/tareas/services/NotificadorService.java, el import **arriba**:

```
import org.springframework.scheduling.annotation.Async;
```

y el método **dentro de la clase**, debajo de notificarSincrono:

```
@Async
public void notificarAsincrono(String destinatario) {
    trabajar(destinatario, "ASINCRONO");
}
```

Y en practica/src/main/java/cl/dgt/tareas/controllers/TramiteController.java, donde dice // escribe aquí:

```
/** Los mismos tres, pero el usuario no espera a ninguno. */
@PostMapping("/asincrono")
public Map<String, Object> asincrono() {
    List.of("ana@sii.cl", "luis@sii.cl",
        ↪ "sofia@sii.cl").forEach(notificador::notificarAsincrono);
    return Map.of("tramite", "creado", "modo", "ASINCRONO");
}
```

Se corre

```
curl -s -o /dev/null -w '%{time_total}s\n' -X POST localhost:8103/tramites/sincrono
curl -s -o /dev/null -w '%{time_total}s\n' -X POST localhost:8103/tramites/asincrono
```

Lo que vas a ver

```
3.011754s    <- síncrono
0.003141s    <- asíncrono
```

Tres segundos contra tres milésimas. Mil veces.

Y **el trabajo se hace igual**: las líneas del notificador siguen saliendo en la consola después de que el curl haya vuelto.

Los dos endpoints se dejan puestos a propósito. El síncrono no sobra: es la mitad de la comparación, y sin él el número queda en una diapositiva y deja de poder comprobarse.

Vas bien si...

El asíncrono responde en milésimas, **y** en la consola siguen apareciendo después las líneas del notificador.

Si te atascas

1 · El asíncrono también tarda 3 segundos.

Tres causas, por frecuencia: falta `@EnableAsync`; el método `@Async` se llama **desde la misma clase** —el problema del proxy—; o `@Async` está sobre un método privado, donde el proxy no llega.

2 · La respuesta vuelve rápido pero no se manda nada.

Mira si hubo una excepción en el hilo de fondo. Con `@Async` y `void`, **la excepción no vuelve a quien llamó**: si nadie la registra, desaparece.

3.4 Paso 4 · Hilos virtuales

Qué vamos a hacer

Cambiar el modelo de hilos del servidor con **una línea de configuración**, y comprobarlo.

Para entenderlo mejor

Los funcionarios de la oficina dejan de ser plantilla fija y pasan a ser gente que aparece cuando hace falta y se va cuando no. La oficina aguanta muchos más ciudadanos a la vez sin contratar a nadie.

El problema

Un hilo de plataforma pesa: un megabyte de pila y un recurso del sistema operativo. Con miles de peticiones concurrentes que se pasan el rato **esperando** —a una base, a otro servicio— tienes miles de hilos caros sin hacer nada.

La alternativa, y por qué no

- **Programación reactiva** (WebFlux): resuelve lo mismo y **cambia cómo se escribe todo el código** — nada de bloquear, todo en cadenas de operadores. Es un cambio de modelo mental.
- **Hilos virtuales**, que es lo de aquí: el código sigue siendo el mismo código bloqueante de siempre, y **una línea** cambia quién lo ejecuta.

Se edita — aquí no se pega nada

En `practica/src/main/resources/application.yml` ya está esto:

```
spring:
  threads:
    virtual:
      enabled: false
```

Cambia false por true. Es todo.

Lo que vas a ver

```
curl -s localhost:8103/tramites/quien
```

```
{"esVirtual":true,
 "hiloQueAtiende":"VirtualThread[#70,tomcat-handler-2]/runnable@ForkJoinPool-1-
↪ worker-2",
 "instancia":"instancia-8103"}
```

Antes decía `Thread[#58,http-nio-8103-exec-1,5,main]` y `esVirtual: false`. Una línea de configuración, y Tomcat entero pasó a hilos virtuales. **No se tocó una línea de código.**

Vas bien si...

`esVirtual` es `true` y el nombre del hilo empieza por `VirtualThread`.

Si te atascas

1 · Sigue diciendo `esVirtual: false`.

No reiniciaste, o la línea quedó mal indentada y la propiedad no se leyó. `virtual` va bajo `threads`, y `threads` bajo `spring`.

3.5 Paso 5 · El problema que nadie ve venir

Qué vamos a hacer

Entender qué pasa con las tareas programadas cuando la aplicación **corre dos veces**. Este paso es de leer y pensar.

Para entenderlo mejor

La DGT abre una segunda sede para atender más público. Las dos sedes tienen el mismo manual, y el manual dice «*a las 2 de la mañana se hace el cierre*».

A las 2 de la mañana se hace el cierre dos veces.

El problema

`@Scheduled` dispara **en cada instancia**. Es exactamente lo que quieres para escalar el tráfico HTTP, y exactamente lo que no quieres para un proceso de cierre: dos cierres simultáneos sobre las mismas filas es un problema de datos.

Y es un problema que **no aparece en desarrollo**, donde siempre corre una sola instancia. Aparece el día del segundo despliegue.

La alternativa, y por qué no

- **Que sólo una instancia tenga las tareas activas** (un perfil): simple, y esa instancia pasa a ser un punto único de fallo — si se cae, no hay cierre.
- **Un candado en la base**, como el del Lab 07: la instancia que lo consigue ejecuta; las demás pasan de largo. Barato, y ya tienes la base.
- **Un planificador con estado compartido** (Quartz en modo clúster, ShedLock): resuelve esto exactamente, y es otra pieza que operar.

Ninguna se implementa hoy, y conviene saber por qué: el objetivo del paso es que reconozcas el problema. Lo peor que puede pasar con las tareas programadas es desplegar la segunda instancia sin haber pensado en esto.

Se comprueba

Puedes verlo tú mismo: arranca **la solución además de tu práctica** —usan puertos distintos— y mira las dos consolas. Las dos imprimen [CIERRE], cada una por su cuenta.

```
[CIERRE] instancia-8103 · vuelta 1 ...  
[CIERRE] instancia-8104 · vuelta 1 ...
```

Dos instancias, dos cierres. Ese instancia-XXXX está en la salida justo para esto.

Vas bien si...

Puedes explicar por qué dos instancias hacen el cierre dos veces, y nombrar al menos dos formas de evitarlo.

4 Lo que aprendiste

1 · fixedDelay y fixedRate no son lo mismo, y el defecto correcto es fixedDelay.

Lo viste en los segundos: 15, 21, 27 — seis, no cinco. Con fixedRate, el día que la tarea tarde más que el intervalo, se solapan.

2 · Un cron sin zona es un error esperando al despliegue.

Seis campos, no cinco, y la zona escrita. Sin ella, la hora la decide el servidor.

3 · @Async saca el trabajo del camino del usuario — y funciona por un proxy.

3,011 s contra 0,003 s. Y llamarlo desde la misma clase lo convierte en síncrono **sin avisar**.

4 · Las tareas programadas se disparan en todas las instancias.

Es el problema que no aparece en tu máquina y aparece en el segundo despliegue. Reconocerlo vale más que cualquiera de las tres soluciones.

5 Para profundizar

- **Cambia fixedDelay por fixedRate** y sube el Thread.sleep a 7 segundos. Mira los timestamps: verás dos ejecuciones solapadas.
- **Quita el zone** del cron y compara con la hora de tu reloj.
- **Llama a notificarAsincrono desde otro método de la misma clase** y mide. Vas a ver los 3 segundos otra vez: es el proxy.
- **Haz que el método @Async lance una excepción** y mira si te enteras. Después cámbialo para que devuelva CompletableFuture y prueba otra vez.
- **Arranca práctica y solución a la vez** y mira los dos [CIERRE].

6 Antes de cerrar

Párala con Ctrl+C — y si arrancaste también la solución, para las dos.

```
./mvnw clean
```

Lo que te llevas:

@Scheduled para el trabajo que ocurre solo, con fixedDelay por defecto y cron con zona cuando importa la hora. @Async para el trabajo que no debe hacer esperar — y ojo con el proxy. Y todo esto se dispara en cada instancia.

Lo que queda pendiente, y abre el Lab 13: tu aplicación funciona **en tu máquina**. Para que funcione en otra hay que llevarla entera: el código, sus dependencias, su Java y su configuración. En el Lab 13 se empaqueta en una imagen — sin Docker.